Algoritmos – 1er Parcial

Propuestas para el 1er Parcial

Fecha límite de entrega: viernes 9/10/2026 23:59 hs

Consigna

Estos son los posibles temas que les pueden ser asignados. Dentro de este conjunto de temas se encuentra el que le fue asignado a usted.

Escriba y resuelva en un archivo `.gbs` (respetando las buenas prácticas) la consigna del tema que le tocó.

Además, proponga usted un procedimiento o una función que modele algún aspecto no contemplado por la consigna relativo al dominio de su enunciado. Se valorará la creatividad y complejidad de la tarea propuesta (la cual debe proponer y resolver). Argumente en qué se basó para escoger su propuesta.

1. El Corral de Ovejas

Con Gobstones podemos modelar el estado de un campo y la organización de un rebaño de ovejas en un tablero genérico. Supongamos que decidimos representar a los elementos del campo con identificadores numéricos basados en bolitas verdes: 1 bolita verde para las ovejas comunes, 2 bolitas verdes para los carneros y 5 bolitas verdes para los perros pastores. Diferenciaremos a las ovejas o carneros enfermos de los sanos mediante la presencia de una bolita roja. Naturalmente, la ausencia de este color marca que el animal está sano. Adicionalmente, el corral principal está identificado en la celda donde hay una bolita azul, y se conviene que la entrada al corral siempre se realiza desde la celda contigua al Sur.

1. Se pide que escriba el procedimiento `MoverOvejaEnfermaAlCorralHacia_` que, asumiendo que se está sobre una celda con una oveja enferma, la traslada hacia la celda contigua en la dirección dada siempre y cuando sea hacia la entrada del corral, dejando a la oveja en dicha celda. Se asume que existe una celda válida hacia `direccionAlCorral` y que contiene la entrada al corral.
2. Definir `cantidadDeAnimales` que indica la cantidad de animales de un tipo dado en todo el campo según su estado de salud (Sano o Enfermo). La cantidad y tipo de parámetros para esta función se deja a cargo del programador. Se tendrán en cuenta las buenas prácticas de programación.
3. Definir la función `cantidadDeParcelasLibres`, que devuelva la cantidad de parcelas (celdas) de todo el campo en las que no hay ningún animal ni estructura.
4. Definir la función `cantidadDeOvejasEnfermas`, que devuelva la cantidad total de ovejas enfermas presentes en todo el campo.
5. Definir el procedimiento `IrAParcelaSinCorralMasAlOeste`, que deje el cabezal en la primera parcela más al Oeste que no contenga la entrada al corral, en la posición más al Sur de esa columna.

2. Logística y Camión de Reparto

Con Gobstones podemos modelar la distribución de carga en el mapa de un depósito logístico sobre un tablero genérico. Supongamos que decidimos representar los distintos tipos de paquetes con un identificador numérico dado por la cantidad de bolitas verdes: 1 para paquetes livianos, 3 para paquetes medianos y 10 para paquetes de carga pesada. Diferenciaremos las celdas que contienen un camión de reparto activo de aquellas con carga en espera mediante la presencia de una bolita negra para indicar la ubicación del camión. La ausencia de este color indica que es un punto de acopio común. Además, las zonas marcadas como punto de entrega prioritario contendrán una bolita azul. También vamos a convenir que los camiones de reparto siempre descargan hacia el Este.

1. Se pide que escriba el procedimiento `DescargarPaquetePesadoEnDestinoHacia_` que, asumiendo que el cabezal está sobre un camión de reparto cargado con paquetes pesados, traslada y descarga un paquete pesado en la zona de entrega prioritaria más próxima hacia la dirección dada, dejando al camión ubicado en dicha celda. Se asume que existe una zona de entrega hacia `direccionDeEntrega`.
2. Definir `cantidadDePaquetes` que indica la cantidad total de paquetes de un tipo determinado almacenados en el depósito, según pertenezcan a un camión activo o a un punto de acopio común. La cantidad de parámetros para esta función se deja a cargo del programador. Se tendrán en cuenta las buenas prácticas.
3. Definir la función `cantidadDePasillosVacios`, que devuelva la cantidad de columnas del depósito logístico que no contienen ningún camión ni paquete acumulado.
4. Definir la función `cantidadDePaquetesPesadosEnEspera`, que devuelva la cantidad total de paquetes pesados que están en puntos de acopio (sin camión) en todo el depósito.
5. Definir el procedimiento `IrAEntregaPrioritariaMasAlEste`, que deje el cabezal en la primera zona de entrega prioritaria más al Este del mapa, ubicada en la celda más al Norte de su correspondiente columna.

3. El Jardín y la Cosecha

Con Gobstones podemos modelar el cultivo y riego de parcelas en un jardín botánico sobre un tablero genérico. Supongamos que decidimos representar el estado de crecimiento de las plantas con un identificador numérico según la cantidad de bolitas verdes: 1 para brotes jóvenes, 3 para plantas en crecimiento y 5 para plantas con frutos maduros listos para cosechar. Diferenciaremos las plantas que sufrieron un ataque de plaga mediante la presencia de una bolita negra. La ausencia de este color indica que la planta se encuentra en estado saludable. Asimismo, el nivel de riego en la parcela actual se mide a través de la presencia de bolitas azules, donde cada bolita representa un litro de agua acumulado. Convenimos que el sistema de riego automático avanza pulverizando hacia el Norte.

1. Se pide que escriba el procedimiento `TratarPlagaConRiegoHacia_` que, asumiendo que el cabezal se encuentra sobre una parcela con plaga, aplica un tratamiento de riego trasladando el agua hacia la planta afectada más próxima en la dirección dada y eliminando la plaga. Se asume que hay una planta con plaga en la dirección `direccionATratar`.
2. Definir `cantidadDePlantas` que indica la cantidad de plantas en un estado de crecimiento dado dentro del jardín botánico, diferenciando si están saludables o afectadas por plagas. La cantidad de parámetros queda a criterio del programador aplicando buenas prácticas.
3. Definir la función `cantidadDeSectoresSinCultivar`, que devuelva la cantidad de columnas de la parcela agrícola en las que no hay ningún tipo de planta ni riego.
4. Definir la función `cantidadDePlantasPlagadas`, que devuelva la cantidad total de plantas afectadas por plagas en todo el jardín botánico.

5. Definir el procedimiento `IrAParcelaConFrutosMasAlSur`, que deje el cabezal en la primera parcela con plantas que tengan frutos maduros listos para cosechar más al Sur-Oeste del tablero.

4. Pac-Man en el Laberinto

Con Gobstones podemos modelar una partida del clásico arcade Pac-Man dentro de un tablero genérico. Supongamos que decidimos representar los elementos móviles e inmóviles del laberinto mediante un identificador numérico dado por la cantidad de bolitas verdes: 1 para los puntos / píldoras pequeñas, 5 para las superpíldoras y 20 para el Pac-Man. Diferenciaremos la presencia de los fantasmas mediante la inclusión de una bolita roja en la celda correspondiente. Por otro lado, la presencia de una bolita negra marca un bloque de pared infranqueable; naturalmente, la ausencia de la bolita negra indica que la casilla es un pasillo libre por donde se puede transitar. Convenimos además que el personaje siempre intenta desplazarse preferentemente hacia el Este.

1. Se pide que escriba el procedimiento `ComerSuperpildoraConFantasmaHacia_` que, asumiendo que se está posicionado sobre Pac-Man, avanza y consume la superpíldora más próxima en la dirección dada, incluso si en esa celda se encuentra un fantasma. Se asume que existe una superpíldora hacia `direccionAComer`.
2. Definir `cantidadDeElementos` que indica la cantidad de elementos de un tipo dado (puntos, superpíldoras o Pac-Man) presentes en el laberinto, según estén amenazados por la presencia de un fantasma o libres de peligro. Diseñar la firma de la función según las buenas prácticas.
3. Definir la función `cantidadDePasillosVacios`, que devuelva la cantidad de columnas del laberinto que no contienen puntos, superpíldoras ni personajes (solo pasillos sin elementos).
4. Definir la función `cantidadDeFantasmasLibres`, que devuelva la cantidad total de fantasmas en el tablero que no están en la misma celda ni contiguos a una superpíldora.
5. Definir el procedimiento `IrAPacManMasAlOeste`, que deje el cabezal en la ubicación de Pac-Man más próxima al Oeste del laberinto, situándose exactamente sobre la celda que lo contiene.

5. Estacionamiento de Vehículos

Con Gobstones podemos modelar la distribución de vehículos en un estacionamiento multinivel usando un tablero genérico. Supongamos que decidimos representar la categoría de los vehículos alojados mediante la cantidad de bolitas verdes: 1 para motocicletas, 2 para automóviles particulares y 4 para camionetas o furgonetas. Diferenciaremos los lugares asignados o reservados exclusivamente para personas con movilidad reducida mediante la presencia de una bolita azul. La ausencia de dicho color indica que la plaza es de estacionamiento general. Adicionalmente, las celdas que contienen una columna o estructura fija del edificio se representan mediante la presencia de una bolita negra. Vamos a convenir que la barrera de ingreso y control de cobro se ubica siempre en la celda más al Sur-Oeste del tablero.

1. Se pide que escriba el procedimiento `ReubicarVehiculoEnLugarReservadoHacia_` que, asumiendo que se está sobre un vehículo estacionado en un lugar común, lo desplaza a la plaza reservada para movilidad reducida más próxima en la dirección dada. Se asume que existe un lugar reservado disponible hacia `direccionAlLugarReservado`.
2. Definir `cantidadDeVehiculos` que indica la cantidad de vehículos de una categoría dada (motocicletas, automóviles o camionetas) estacionados en el edificio, filtrados por el tipo de plaza que ocupan (General o Reservada). La cantidad de parámetros requeridos se deja a elección del programador considerando buenas prácticas.
3. Definir la función `cantidadDeNivelesVacios`, que devuelva la cantidad de columnas del edificio de estacionamiento que no tienen ningún vehículo aparcado ni columnas de la estructura.
4. Definir la función `cantidadDeVehiculosReservados`, que devuelva la cantidad total de vehículos estacionados en plazas reservadas para movilidad reducida en todo el edificio.
5. Definir el procedimiento `IrAPlazaLibreReservadaMasAlEste`, que deje el cabezal en la primera plaza reservada sin vehículo más próxima al Este del edificio, en la celda más al Sur de esa columna.

Criterio

- Se evaluará el correcto uso de las **buenas prácticas** vistas en la materia y la comunicación de cada código.
- Se podrá usar el material trabajado en clase como la guía de actividades de cada uno.

Entrega

(1) Descargue el archivo `.gbs` con nombre `algoritmos-primer-parcial-1C2026-apellido-nombre.gbs`.

(2) Llene el formulario https://forms.gle/PooMPKWgKrC3hctb6

(3) Envíe su examen a la casilla `javierpedro.garcia@unahur.edu.ar`. En **Asunto** coloque *Primer parcial algoritmos* y en el cuerpo, *Envío primer parcial luego de completar el formulario de entrega.*